```
# IMDB Sentiment Classification using Deep Neural Network
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import layers, models #Y

import pandas as pd

df = pd.read_csv(
   'IMDB_Dataset.csv',
   encoding='latin-1',
   engine='python',
   on_bad_lines='skip'
)
print(df.head())
print(df['sentiment'].value_counts()) #Y

# Convert labels: 'positive' -> 1, 'negative' -> 0
le = LabelEncoder()
y = le.fit_transform(df['sentiment'])     # Encodes text labels to numbers
texts = df['review'].values               # Raw text reviews #Y

# Tokenization -- convert words to numbers
max_words = 10000                         # Only top 10,000 words
max_len   = 200                           # Max words per review
tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(texts)             # Build vocabulary from text
X = tokenizer.texts_to_sequences(texts)   # Convert text to number sequences
X = pad_sequences(X, maxlen=max_len)      # Pad shorter reviews with 0s #Y

# Split data
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42) #Y

# Build the neural network
model = models.Sequential([
  # Embedding: converts word indices into dense vectors
  layers.Embedding(max_words, 32, input_length=max_len),
  # Flatten the 2D embedding into 1D for Dense layers
  layers.GlobalAveragePooling1D(),
  layers.Dense(64, activation='relu'),    # Hidden layer
  layers.Dropout(0.5),                    # Randomly drops 50% neurons (prevents overfitting)
  layers.Dense(1, activation='sigmoid')  # Output: probability 0-1 (sigmoid for binary)
]) #Y

# Compile model for binary classification
model.compile(
  optimizer='adam',
  loss='binary_crossentropy',           # Loss for binary (0/1) problems
  metrics=['accuracy']                  # We care about % correct
) #Y

model.summary()                         # Print model architecture
```

```
# Train the model
history = model.fit(
  X_train, y_train,
  epochs=5,                    # 5 passes (text training is slower)
  batch_size=64,
  validation_split=0.1,
  verbose=1
) #Y

# Evaluate on test data
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {acc*100:.2f}%")

# Predict sentiment for a new review
sample = ["This movie was absolutely brilliant and wonderful!"]
seq = tokenizer.texts_to_sequences(sample)
seq = pad_sequences(seq, maxlen=max_len)
pred = model.predict(seq)[0][0]
print("Positive" if pred > 0.5 else "Negative", f"({pred:.2f})")

model.summary()                # Print model architecture
```